

Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3092 Deep Learning, Semestre II 2026



Laboratorio 6

Roberto Barreda #23354

Guatemala, 29 de septiembre de 2026

1. Investigación: transfer learning en PyTorch

Tabla 1. Herramientas de PyTorch para reutilizar un modelo preentrenado y medir su costo.

Herramienta	Propósito y parámetros relevantes
<code>torchvision.models.vgg16</code> y <code>VGG16_Weights</code>	<code>vgg16(weights=...)</code> construye la red y carga los pesos; con <code>weights=None</code> los pesos son aleatorios. <code>VGG16_Weights.IMAGENET1K_V1</code> guarda los pesos de ImageNet (71.6 % top-1) y sus metadatos. <code>weights.transforms()</code> devuelve el preprocesamiento esperado: redimensionar a 256, recorte central de 224 y normalización con media (0.485, 0.456, 0.406) y desviación (0.229, 0.224, 0.225).
<code>model.features</code> , <code>model.avgpool</code> y <code>model.classifier</code>	<code>features</code> tiene 13 convoluciones 3×3 con ReLU y 5 MaxPool2d que separan 5 bloques. <code>avgpool</code> es <code>AdaptiveAvgPool2d((7, 7))</code> y entrega 512×7×7 con cualquier resolución de entrada. <code>classifier</code> tiene tres capas lineales (25088→4096→4096→1000) con ReLU y Dropout(0.5); la última se reemplaza por <code>Linear(4096, 10)</code> .
<code>requires_grad</code>	Con <code>requires_grad=False</code> autograd no calcula el gradiente del parámetro y el optimizador no lo cambia. Un bloque se congela con <code>bloque.requires_grad_(False)</code> o recorriendo <code>bloque.parameters()</code> , y se descongela con <code>True</code> .
<code>model.train()</code> , <code>model.eval()</code> y <code>torch.no_grad()</code>	<code>train()</code> y <code>eval()</code> cambian Dropout y BatchNorm: en <code>eval</code> Dropout se apaga y BatchNorm usa sus estadísticas acumuladas. <code>torch.no_grad()</code> no construye el grafo de autograd y reduce memoria y tiempo. Son independientes; en evaluación se usan juntos.
<code>param_groups</code> del optimizador	El optimizador recibe una lista de diccionarios con <code>params</code> y sus hiperparámetros (<code>lr</code> , <code>weight_decay</code>). Así el backbone usa un learning rate menor que el clasificador. OneCycleLR recibe un <code>max_lr</code> por grupo.
<code>torchinfo.summary</code>	Ejecuta un forward de prueba y reporta por capa la forma de salida, los parámetros totales y entrenables y las mult-adds (MACs). Parámetros: <code>input_size</code> , <code>col_names</code> , <code>depth</code> y <code>device</code> .
<code>torch.cuda.max_memory_allocated()</code> y <code>torch.cuda.synchronize()</code>	El primero da el pico de memoria ocupada por tensores desde el último <code>reset_peak_memory_stats()</code> . El segundo espera a que la GPU termine sus kernels; como CUDA es asíncrono, se llama antes de leer el reloj.

Una MAC (multiply-accumulate) es una multiplicación seguida de una suma y un FLOP es una sola operación de punto flotante, por lo que $FLOPs \approx 2 \times MACs$. La documentación de torchvision reporta 15.47 GFLOPS para VGG-16, que coinciden con las 15.48 GMACs que calcula torchinfo. En este informe $FLOPs = 2 \times MACs$.

Los parámetros totales son todos los pesos del modelo y definen la memoria y el costo de inferencia. Los entrenables son los que tienen `requires_grad=True` y definen la memoria del optimizador y el costo del backward. Congelar capas reduce los entrenables, pero no cambia los totales ni el costo del forward.

2. Resultados de las iteraciones

Todas las iteraciones usaron AdamW, OneCycleLR, lotes de 128 y precisión mixta bfloat16. Las métricas son de validación (5,000 imágenes) en la epoch con mejor F1 macro. En VGG-16 los bloques que no se listan quedan congelados y la entrada es de 224×224, salvo en FE-3.

Tabla 2. Resultados de validación por iteración. Entren.: parámetros entrenables; s/ep.: segundos por epoch de entrenamiento; Mem.: memoria pico de GPU; aug: augmentation; wd: weight decay; LS: label smoothing; lr bb: learning rate del backbone.

Iter.	Configuración	Entren. (M)	Mejor ep.	Acc.	Prec.	Recall	F1	s/ep.	Total (min)	Mem. (GB)
CNN-1	3 bloques 32-128, sin aug, sin dropout, wd 0, lr 1e-3, 20 ep	0.29	17	0.857	0.858	0.857	0.857	1.8	0.6	0.79
CNN-2	3 bloques 64-256, aug, dropout 0.1/0.3, wd 0.05, lr 2e-3, 30 ep	1.15	28	0.919	0.919	0.919	0.919	2.1	1.0	0.87
CNN-3	4 bloques 64-512, aug, dropout 0.1/0.3, wd 0.05, LS 0.1, lr 2e-3, 40 ep	4.69	36	0.931	0.931	0.931	0.931	2.7	1.8	0.94
FE-1	224×224, clasificador, lr 1e-3, 8 ep	119.59	8	0.904	0.905	0.904	0.904	48.9	6.5	4.34
FE-2	224×224, clasificador, lr 1e-4, 8 ep	119.59	8	0.912	0.912	0.912	0.911	48.9	6.5	4.34
FE-3	112×112, clasificador, lr 1e-4, 8 ep	119.59	8	0.892	0.892	0.892	0.891	19.6	2.6	3.01
FT-1	224×224, bloque 5 + clasificador, lr 1e-4, lr bb 1e-5, 8 ep	126.67	8	0.925	0.925	0.925	0.925	53.3	7.1	4.42
FT-2	224×224, bloques 4-5 + clasificador, lr 1e-4, lr bb 1e-5, 8 ep	132.57	8	0.934	0.934	0.934	0.934	67.7	9.0	4.48
FT-3	224×224, bloques 4-5 + clasificador, lr 1e-4, lr bb 5e-5, 8 ep	132.57	8	0.947	0.947	0.947	0.947	67.5	9.0	4.48
FT-4	224×224, bloques 3-5 + clasificador, lr 1e-4, lr bb 1e-5, 8 ep	134.04	8	0.941	0.941	0.941	0.941	83.5	11.1	4.50

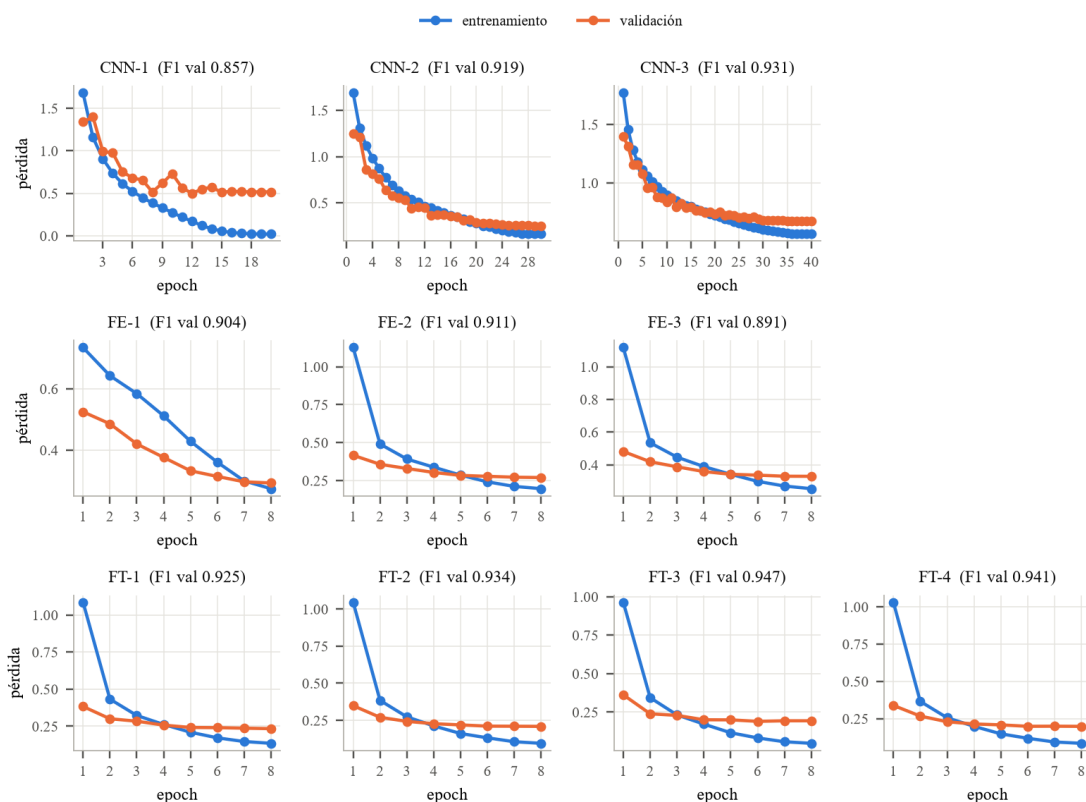


Figura 1. Pérdida de entrenamiento y validación por epoch de las 10 iteraciones. CNN-1, sin regularización, muestra overfitting: su pérdida de entrenamiento llega a 0.02 y la de validación queda en 0.52.

3. Comparación de rendimiento y recursos

Se compara la mejor iteración de cada modelo según el F1 de validación, evaluada una sola vez sobre el test oficial (10,000 imágenes). Hardware: GPU NVIDIA GeForce RTX 4080 Laptop (12 GB), CPU Intel Core i9-13900HX y 16 GB de RAM, con PyTorch 2.13 y CUDA 13.0.

Tabla 3. Rendimiento y recursos de la mejor configuración de cada modelo. Cómputo de entrenamiento = $2 \times (\text{MACs totales} + 2 \times \text{MACs entrenables}) \times 45,000 \text{ imágenes} \times \text{epochs}$, porque el backward solo recorre las capas con parámetros entrenables.

Métrica	CNN desde cero	VGG-16 feature extractor	VGG-16 fine-tuning
Iteración elegida	CNN-3	FE-2	FT-3
Parámetros totales (M)	4.69	134.30	134.30
Parámetros entrenables (M)	4.69	119.59	132.57
GFLOPs por imagen (forward)	0.419	30.960	30.960
Latencia GPU, lote 1 (ms/img)	1.25	1.88	1.62
Tiempo GPU, lote 256 (ms/img)	0.0107	0.8851	0.8843
Tiempo por epoch (s)	2.7	48.9	67.5
Epoch con mejor validación / total	36 / 40	8 / 8	8 / 8
Tiempo total de entrenamiento (min)	1.8	6.5	9.0
Memoria pico de GPU (GB)	0.94	4.34	4.48
Cómputo de entrenamiento (PFLOPs)	2.26	11.32	19.98
Accuracy test	0.9285	0.8991	0.9400
Precision test (macro)	0.9286	0.8988	0.9399
Recall test (macro)	0.9285	0.8991	0.9400
F1 test (macro)	0.9284	0.8987	0.9399
Accuracy test con 10 % de datos	0.7989	0.8451	0.8891
F1 test con 10 % de datos	0.7973	0.8445	0.8887

Con lote de 1 las tres latencias quedan entre 1.2 y 1.9 ms porque domina el costo de lanzar kernels; con lotes de 256 VGG-16 tarda 0.88 ms por imagen y la CNN 0.011 ms. Las dos variantes de VGG-16 tienen el mismo costo de inferencia; la diferencia está en el backward.

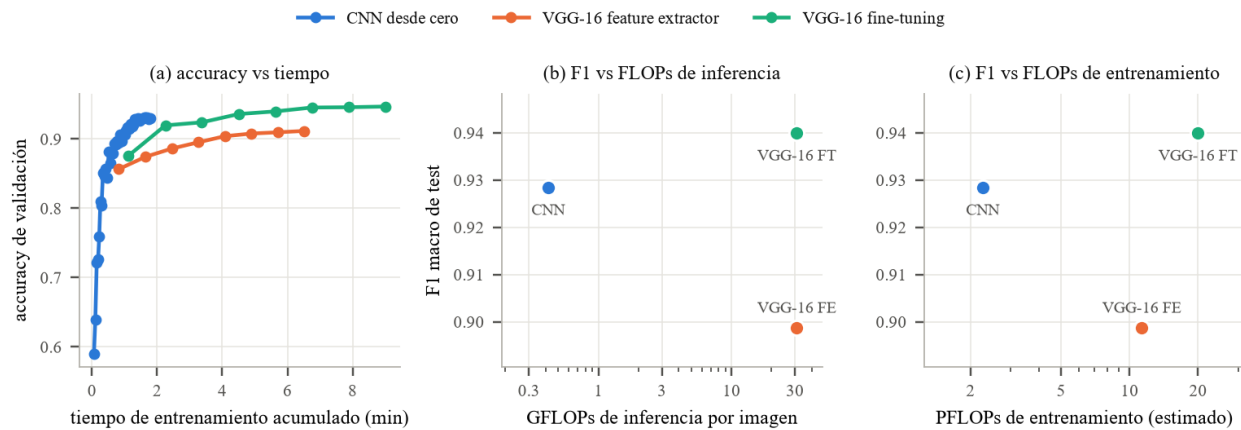


Figura 2. (a) Accuracy de validación contra tiempo de entrenamiento acumulado. (b) F1 de test contra FLOPs de inferencia por imagen. (c) F1 de test contra FLOPs de entrenamiento estimados. Los ejes x de (b) y (c) son logarítmicos.

4. Análisis de resultados

¿Cuál de los tres modelos obtuvo el mejor desempeño en test? ¿Qué tan grande fue la diferencia y justifica el costo adicional en parámetros, FLOPs y tiempo de entrenamiento?

VGG-16 con fine-tuning (FT-3) obtuvo 94.00 % de accuracy y F1 de 0.940. Supera a la CNN desde cero por 1.15 puntos (600 errores contra 715, 16 % menos) y al feature extractor por 4.1 puntos. A cambio usa 29 veces más parámetros, 74 veces más FLOPs por imagen, 5 veces más tiempo de entrenamiento y 83 veces más tiempo de GPU por imagen en lotes. El costo se justifica solo si cada punto de accuracy importa y hay una GPU disponible; en otro caso la CNN desde cero da un resultado cercano con una fracción de los recursos.

¿Qué modelo ofrece la mejor relación entre desempeño y recursos?

La CNN desde cero. En la Figura 2a llega a 93.1 % de validación en 1.6 minutos; el fine-tuning necesita 4.5 minutos para superar ese valor y termina en 9 minutos. En las Figuras 2b y 2c la CNN queda a la izquierda con F1 de 0.928: 0.42 GFLOPs por imagen y 2.3 PFLOPs de entrenamiento, contra 31 GFLOPs y 20 PFLOPs del fine-tuning. El feature extractor queda dominado: tiene el mismo costo de inferencia que el fine-tuning y el F1 más bajo.

VGG-16 fue entrenada con fotografías de 224×224 y CIFAR-10 tiene imágenes de 32×32. ¿Por qué las features preentrenadas siguen siendo útiles? ¿Qué efecto tiene el redimensionamiento en el desempeño y en el costo?

Los primeros bloques detectan bordes, colores y texturas que no dependen del dataset, y los superiores combinan partes de objetos. Las 10 clases de CIFAR-10 tienen equivalentes en ImageNet (perros, gatos, aves, vehículos, barcos), así que esas features ya separan las categorías. Al redimensionar, el objeto queda con una escala parecida a la de ImageNet y el último bloque produce un mapa de 7×7 en lugar de 1×1. La interpolación no agrega información, pero el costo crece con los píxeles: 49 veces más que a 32×32. A 112×112 (FE-3) el costo baja a 3.96 GMACs y el tiempo por epoch de 48.9 a 19.6 s, pero la accuracy de validación cae de 91.2 % a 89.2 %.

En el fine-tuning, ¿cómo afectó el número de bloques descongelados y el learning rate del backbone? ¿Observaron señales de overfitting o de catastrophic forgetting?

Con learning rate de backbone 1e-5, cada bloque descongelado mejoró la validación: 92.5 % con el bloque 5, 93.4 % con los bloques 4-5 y 94.1 % con los bloques 3-5, mientras el tiempo por epoch subió de 53 a 68 y 84 s. Subir ese learning rate a 5e-5 con los bloques 4-5 dio 94.7 %, el mejor resultado, sin costo extra por epoch. Hay señales de overfitting: en FT-3 la pérdida de entrenamiento baja a 0.05 y la de validación se estanca en 0.19 desde la epoch 6. No hubo catastrophic forgetting: la validación subió en cada epoch y desde la primera (87.6 %) superó a la del feature extractor en la misma epoch (85.6 %).

Con base en el experimento de la sección 4.1, ¿cuál modelo se vio más afectado al reducir los datos? ¿Cómo se relaciona esto con lo discutido en clase sobre cuándo usar feature extraction, fine-tuning o entrenar desde cero?

La CNN desde cero: con 4,500 imágenes su F1 de test bajó de 0.928 a 0.797 (13.1 puntos). El feature extractor bajó 5.4 puntos (0.899 a 0.845) y el fine-tuning 5.1 (0.940 a 0.889). Con pocos datos el feature extractor supera a la CNN por 4.7 puntos, al revés que con el dataset completo. Esto coincide con lo visto en clase: con pocos datos y un dominio parecido al de preentrenamiento conviene feature extraction o fine-tuning con learning rate bajo; con más datos se pueden descongelar más capas; entrenar desde cero requiere muchos datos y con 45,000 imágenes ya es competitivo.

Analicen las matrices de confusión: ¿qué pares de clases se confunden más en cada modelo? ¿Los tres modelos cometen los mismos errores?

Gato-perro es el par más confundido en los tres modelos (142 errores en la CNN, 198 en el feature extractor y 138 en el fine-tuning), seguido de automóvil-camión (45, 78 y 54). Después cambian: la CNN confunde gato-rana (42) y pájaro-gato (41); el feature extractor, pájaro-venado (65) y gato-rana (59); el fine-tuning, avión-barco (35) y pájaro-gato (33). Los tres fallan en los pares con forma y textura parecidas que se anticiparon en la exploración, pero con frecuencias distintas: el feature extractor suma 1,009 errores, la CNN 715 y el fine-tuning 600.

Si tuvieran que desplegar un modelo para este problema en (a) un servidor con GPU y (b) un dispositivo móvil, ¿qué modelo elegirían en cada caso y por qué? ¿Qué arquitectura preentrenada alternativa a VGG-16 considerarían para el caso (b)?

En el servidor con GPU elegiría VGG-16 con fine-tuning: tiene el mejor F1 (0.940), tarda 1.6 ms por imagen y sus 134 M de parámetros (537 MB en float32) caben en la GPU. En el móvil elegiría la CNN desde cero: 4.7 M de parámetros (19 MB en float32), 0.42 GFLOPs por imagen y 1.15 puntos menos de accuracy. Como alternativa preentrenada para el móvil consideraría MobileNetV3-Large, que usa convoluciones depthwise separables, tiene 5.5 M de parámetros y 0.22 GMACs a 224×224, y permite hacer transfer learning con un costo que el dispositivo soporta.

5. Conclusiones

1. VGG-16 con fine-tuning de los bloques 4 y 5 fue el mejor modelo en test, con 94.0 % de accuracy y F1 macro de 0.940.
2. La CNN desde cero llegó a 92.9 % en test con 29 veces menos parámetros y 74 veces menos FLOPs por imagen que VGG-16.
3. VGG-16 como feature extractor obtuvo 89.9 % en test y quedó por debajo de la CNN desde cero con el dataset completo.
4. Con el 10 % de los datos, la CNN desde cero perdió 13.1 puntos de F1 y los modelos basados en VGG-16 perdieron cerca de 5.
5. En el fine-tuning, subir el learning rate del backbone de 1e-5 a 5e-5 aportó más que descongelar el bloque 3 y no aumentó el tiempo por epoch.

Enlace al repositorio: <https://github.com/bar23354/DL-Lab06.git>